

사용 매뉴얼 · 설치를 마친 다음에 보는 안내서

# 회사 AI, 이렇게 쓰면 됩니다

설치는 끝났습니다. 이제 어떻게 대화하고, 검증된 프롬프트를 어떻게 꺼내 쓰고, 좋은 노하우를 어떻게 주고받는지 그 그림과 함께 설명합니다. 명령어를 외울 필요는 없습니다.



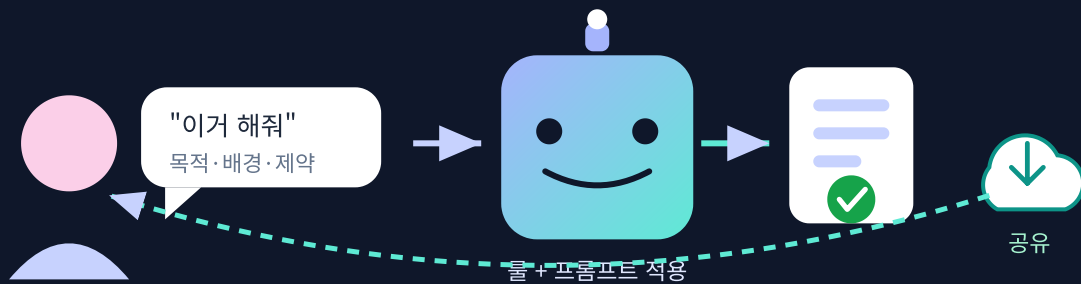
에이전트 라이프사이클



대화 요령 · 프롬프트



지식 공유 (가져오기 · 승격 · 내보내기)

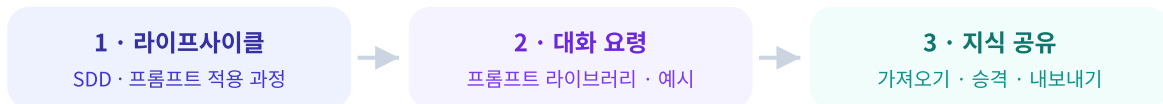


좋은 결과는 다시 회사 지식으로 — 가져오기 · 승격 · 내보내기

## 0 이 매뉴얼을 보는 법

설치 안내서( `INSTALL_GUIDE.html` )로 설치를 마쳤다면, 이 문서는 “이제 어떻게 쓰지?”에 답합니다.

기억할 것은 세 가지입니다 — ① 에이전트가 어떻게 일하는지 알고, ② 잘 부탁하는 요령을 익히고, ③ 좋은 결과를 회사 지식으로 주고받는 것입니다.



### ✓ 한 가지만 기억하세요

에이전트는 “회사 룰을 먼저 읽고, 적절한 깊이를 스스로 정해서, 결과를 보고”합니다. 여러분이 **목적·배경·제약**만 분명히 말하면 나머지는 에이전트가 알아서 맞춰 줍니다.

① 라이프사이클

SDD 소개

프롬프트 적용 과정

② 대화 요령

프롬프트 라이브러리

③ 지식 공유

가져오기

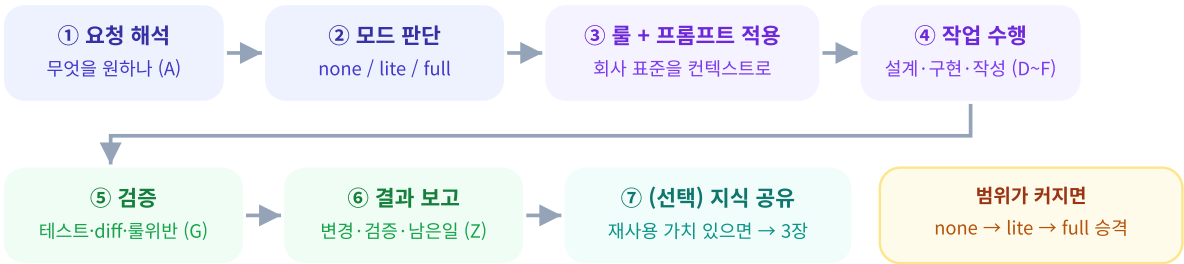
승격

내보내기

꼭 지킬 것

# 1 에이전트의 라이프사이클

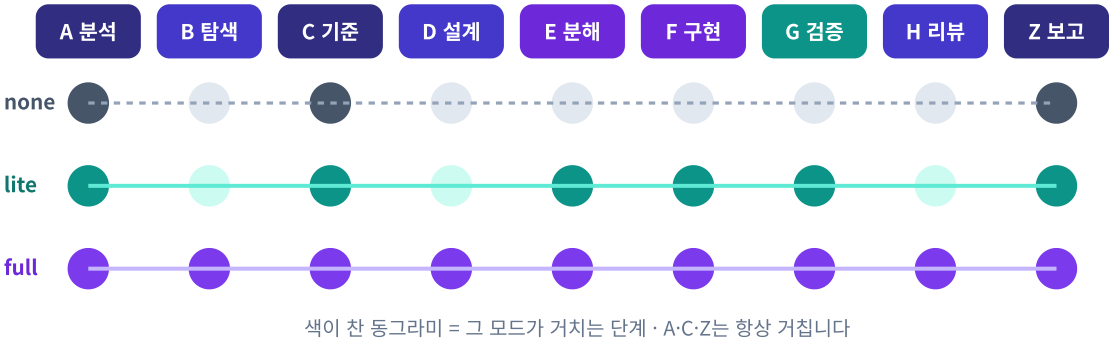
요청 한 건이 결과로 나오기까지, 에이전트 안에서는 정해진 흐름이 돕니다. 이걸 알면 “왜 이렇게 답하지?”가 이해됩니다.



한 건의 요청이 도는 흐름 — 단계 기호(A~Z)는 아래 SDD 소개에서 이어집니다.

## ● SDD 소개 — 같은 흐름, 다른 깊이

SDD(Spec Driven Development)는 “문서부터 쓰는 무거운 방법론”이 아닙니다. **매번 질문으로 멈추지 않도록**, 에이전트가 요청의 무게를 보고 **스스로 적절한 깊이**를 고르는 규칙입니다. 세 모드는 **다른 방법이 아니라 같은 A~Z 흐름**을 얼마나 압축하느냐의 차이입니다.



| 모드   | 흐름                       | 언제                           | 산출물              |
|------|--------------------------|------------------------------|------------------|
| none | A → C → Z                | 단순 질문, 오타·표현 수정처럼 영향 없는 작은 일 | 별도 문서 없음. 결과 보고만 |
| lite | A → C → E → F<br>→ G → Z | 한두 파일 수준 문서/설정 보강, 명확한 작은 변경 | 간단한 계획 + 변경 후 확인 |

| 모드   | 흐름                                      | 언제                                  | 산출물                                          |
|------|-----------------------------------------|-------------------------------------|----------------------------------------------|
| full | A → B → C → D<br>→ E → F → G →<br>H → Z | 새 기능·구조 변경,<br>여러 사람이 이어받<br>는 큰 작업 | requirements / design /<br>tasks / review 스펙 |

### 📌 내가 직접 모드를 정하고 싶다면

그냥 말로 지정하면 됩니다. 💬 **이건 가볍게 lite로 빠르게 해줘**

💬 **설계 문서까지 남기는 full로 진행해줘** 지정하지 않으면 에이전트가 알아서 고르

고, 작업 중 범위가 커지면 스스로 승격합니다. (단, 파괴적 작업·보안·비용·외부 배포는 모드와 무관하게 먼저 확인합니다.)

## ● 프롬프트가 적용되는 과정 — 층층이 쌓입니다

에이전트가 보는 “지시”는 내 한 줄 요청만이 아닙니다. 회사 표준이 여러 겹으로 미리 깔려 있고, 그 위에 내 요청이 얹힙니다. 그래서 같은 한 줄이라도 회사 룰에 맞는 결과가 나옵니다.



아래에서 위로 쌓인 층이 한꺼번에 에이전트의 “컨텍스트”가 됩니다. 진입점은 룰을 직접 담지 않고 `.agents/rules/` 를 가리키는 포인터입니다.

**이게 왜 좋은가요?** 매번 “보안 지켜”, “출처 남겨”, “커밋은 회사 양식으로”를 말하지 않아도 됩니다. 이미 룰 층에 깔려 있어서, 여러분은 **하고 싶은 일만** 말하면 됩니다.

## 2 에이전트와의 대화 요령

좋은 결과의 80%는 “잘 부탁하기”에서 나옵니다. 어렵지 않습니다 — 네 가지만 챙기면 됩니다.

 **목적**  
무엇을 원하나

 **대상**  
누가 읽나

 **배경**  
상황·입력 자료

 **제약·출력**  
조건·원하는 형식

### 막연한 요청

 보고서 좀 써줘

→ 무슨 보고서인지, 누구에게, 무슨 내용인지 몰라 되묻거나 엉뚱한 결과가 나옵니다.

### 좋은 요청

 이번 주 작업 로그를 팀장 보고용 주간보고로 정리해줘. 성과·리스크·다음주 계획 3단락, 표 없이.

→ 목적·대상·배경·제약·형식이 모두 들어 있어 한 번에 쓸 만한 결과가 나옵니다.

### 대화를 더 잘하는 습관

- ① 한 번에 완벽을 노리지 말기 — 초안을 받고 “이 부분 더 짧게”처럼 이어서 다듬으세요.
- ② 예시를 주기 — “이런 톤으로”라며 샘플을 붙이면 정확도가 올라갑니다.
- ③ 민감정보는 가리기 — 고객명·번호는 `<고객명>` 처럼 placeholder로.

## ● 프롬프트 라이브러리 사용법

매번 “좋은 요청”을 새로 짜기는 번거롭습니다. 그래서 설치 시 검증된 프롬프트 골격 묶음(프롬프트 라이브러리)이 함께 깔립니다. 직무에 맞는 골격을 꺼내 빈칸만 채우면 됩니다.

설치된 위치는 사용하는 AI 도구에 따라 다릅니다(의미는 같습니다).

Codex · `.agents/prompts/`

Claude · `.claude/prompts/`

Gemini · `.gemini/prompts/`

Cursor · `.cursor/prompts/`

Kiro · `.kiro/prompts/`

기본으로 들어오는 골격(예시):

| 파일                                                                                           | 용도                                    | 직무        |
|----------------------------------------------------------------------------------------------|---------------------------------------|-----------|
| <code>_universal-template.md</code>                                                          | 모든 프롬프트의 공통 골격<br>(역할·목적·대상·배경·제약·출력) | 전 직무      |
| <code>dev-implementation.md</code>                                                           | 구현 요청 → 설계·코드·설명·개선점                  | developer |
| <code>doc-writing.md</code> · <code>comparison-analysis.md</code> · <code>research.md</code> | 문서 작성 · 비교 분석 · 리서치                   | 전 직무      |
| <code>product/</code> (prd · user-story · prioritization)                                    | PRD 초안 · 사용자 스토리 · 우선순위               | product   |
| <code>business/</code> (faq · proposal · announcement)                                       | FAQ · 제안 메시지 · 공지문                    | business  |

꺼내 쓰는 방법은 두 가지 — 둘 다 말로 시키면 됩니다.

1

## 이름으로 불러오기

어떤 골격이 있는지 모르면 먼저 목록을 물어보고, 마음에 드는 걸 채워 달라고 합니다.

💬 내 프롬프트 라이브러리에 뭐가 있는지 보여줘.

💬 dev-implementation 골격으로 이 요구사항을 채워서 구현해줘.

2

## 공통 골격으로 새로 만들기

딱 맞는 게 없으면 `_universal-template.md` 의 6개 섹션(역할·목적·대상·배경·제약·출력)을 기준으로 만들어 달라고 합니다.

💬 범용 템플릿 골격으로, 신규 거래처 안내 메일 프롬프트를 만들어줘.

## ● 프롬프트 예시

실제 골격 하나를 보면 감이 옵니다. 아래는 `dev-implementation.md` 의 구조입니다 — **대괄호 부분만** 채우면 됩니다.

## <> 개발 구현 프롬프트 (dev-implementation)

당신은 Senior Software Engineer이다.  
다음 요구사항을 구현한다.

### ## 요구사항

[구현할 기능 - 민감정보는 placeholder로]

### ## 개발환경

- OS / Language / Framework / Compiler

### ## 조건

- 유지보수성 · 성능 · 예외처리 · 주석 포함

### ## 출력

1. 설계    2. 코드    3. 설명    4. 개선점

직무별로 자주 쓰는 한 줄 요청 예시:

| 직무        | 이렇게 말하면 됩니다                                                        |
|-----------|--------------------------------------------------------------------|
| 개발        | 💬 dev-implementation 골격으로 “CSV 업로드 검증” 기능을 Node로 구현해줘. 예외처리·주석 포함. |
| 기획<br>·PM | 💬 product/prd 골격으로 “알림 설정 화면” PRD 초안 잡아줘. 대상은 모바일 사용자.             |
| 사업<br>·CS | 💬 business/faq 골격으로 환불 정책 FAQ 5개 만들어줘. 고객명은 <고객명>으로.               |
| 공통        | 💬 comparison-analysis 골격으로 A안·B안을 표로 비교하고 결론 내줘.                   |

✅ 잘 통한 프롬프트는 버리지 마세요

“이거 좋다” 싶은 프롬프트가 있으면 **3장의 내보내기**로 회사 라이브러리에 올릴 수 있습니다.  이 프롬프트 자산화해줘 라고 말하면 에이전트가 정리해 줍니다.



### 3 지식 공유 방법

남이 만든 좋은 프롬프트·스킬을 가져오고, 검증된 건 승격하고, 내 노하우는 내보냅니다. 이 순환이 회사 AI 자산을 키웁니다.



핵심 원칙: 외부에서 받은 건 절대 바로 회사 자산처럼 쓰지 않습니다. 먼저 검역 구역에 두고, 검증된 것만 승격합니다.

#### ● 가져오기 — 다운로드 / MD 복사

다른 사람이 공유한 프롬프트·스킬 `.md` 를 내 프로젝트로 가져오는 단계입니다. 어떻게 받든 먼저 검역 구역 (`.agents/externals/`)에 두고, 보안검사를 거친 뒤에만 실제로 씁니다.

1

#### 어디서 받을지 고르기

공유 폴더 Google Drive AI-Knowhow/ (skills · prompts)에서 받거나, GitHub prompts / skills 후보 브랜치에서 받습니다.

💬 AI-Knowhow/prompts 폴더의 .md 목록을 보여줘.

2

#### 받는 방법 — ① 다운로드 ② MD 복사

① 직접 다운로드(권장): AI 도구에 Google Drive 커넥터가 연결돼 있으면 에이전트가 `.md` 를 원형 그대로 내려받습니다. 연결이 없으면 사람이 Drive 웹에서 내려받아 `.agents/externals/` 에 넣습니다.

② MD 복사: 짧은 내용은 `.md` 본문을 복사해 붙여넣어도 됩니다. (큰 파일을 base64 등으로 옮겨 적는 방식은 피하세요.)

3

#### 검역 구역에 두기

받은 파일은 곧장 쓰지 않고 아래 위치에 둡니다 — 여긴 “검증 전 공유본” 보관소입니다.

.agents/externals/skills/

.agents/externals/prompts/

.agents/externals/docs/

## 4

### 보안검사 받기

에이전트가 로컬 파일에서 **민감정보·credential·고객 데이터·출처** 누락을 점검합니다. 필요하면 가린(redacted) 사본을 만듭니다.

💬 externals에 받은 이 .md를 보안검사하고, 무슨 자산인지·어디 들어갈지·기존 파일과 충돌하는지 알려줘.

## 5

### 확인 후 배치

요약·배치 위치·충돌 여부를 확인하고 컨펌하면, 충돌 시 기본은 **add-namespaced**(이름공간을 나눠 나란히 두기)로 안전하게 배치합니다. (다른 선택지: keep-existing · rename-proposed · replace-existing)

#### ⚠️ 검역이 필요한 이유

외부 .md 에는 남의 민감정보가 섞여 있거나, 우리 프로젝트와 다른 가정이 들어 있을 수 있습니다. .agents/externals/ 는 **검토 전 자료가 실수로 커밋되지 않도록** 보호되는 격리 구역입니다.

### ● 승격 및 사용 방법

검역 구역의 자료가 **여러 사람·여러 작업에서 반복적으로 잘 쓰이면**, 운영자 검토를 거쳐 **공식 자산**으로 승격합니다. 승격되면 카탈로그/프로필에 들어가 설치만 하면 누구나 “정식 자산”으로 받게 됩니다.

| 상태   | 위치                                 | 의미                        |
|------|------------------------------------|---------------------------|
| 검증 전 | .agents/externals/...              | 받아만 둔 공유본. 내 프로젝트에서 시험 사용 |
| 후보   | GitHub prompts / skills 브랜치        | 공유 후보로 수집됨. 아직 공식 아님      |
| 공식   | main · company-* · catalog/profile | 검증 통과. 설치 기본 자산으로 배포      |

승격 게이트(기본값 — governance가 바뀌지 않는 한):

사용 ≥ 5회

사용자 ≥ 2명

성공률 ≥ 80%

표현 안정

owner·reviewer

승격은 에이전트가 단독으로 “공식”이라 선언하지 않습니다. 게이트 충족 + **운영자 수동 검토**를 거칩니다.  이 프롬프트, 승격해도 될지 게이트 기준으로 점검하고 승격 제안 정리해줘.

### ⚠ 승격 없이 그냥 쓰면 — 한계점

검역 구역의 자료를 시험 삼아 쓰는 건 괜찮습니다. 다만 **승격 전**에는 “**검증 전 공유본**”이라는 점을 잊으면 안 됩니다.

#### 자동 배포 안 됨

설치/업데이트 대상이 아니라, 새 프로젝트나 동료에게 그대로 따라오지 않습니다. 매번 수동으로 다시 가져와야 합니다.

#### 품질 보증 없음

회사 검증(보안·출처·중복·품질)을 통과하지 않았습니다. 잘못된 가정이나 빠진 출처가 있을 수 있습니다.

#### 책임자 불명확

owner/reviewer가 없어, 문제가 생기거나 업데이트가 필요할 때 누구에게 물어야 할지 불분명합니다.

#### 표류·중복 위험

여러 사람이 각자 조금씩 고친 사본이 흩어져, 어느 게 “맞는 버전”인지 알 수 없게 됩니다 (drift).

한 줄 요약: **혼자 잠깐 쓰는 건 OK, 팀 표준으로 삼으려면 반드시 승격**. 적용 시 에이전트는 항상 “**검증 전 공유본**”이라고 표시하고 컨펌을 받습니다.

## ● 내보내기 — 업로드 / MD 추출

내가 잘 쓴 프롬프트·스킬·노하우를 회사 공유 채널로 올리는 단계입니다. 에이전트가 먼저 **정제(민감정보 제거·출처·frontmatter)**를 도와준 뒤 올립니다.

1

### 자산화 요청 (MD 추출)

대화에 흩어진 내용을 재사용 가능한 `.md` 한 장으로 뽑아냅니다.

 방금 그 프롬프트, 공유할 수 있게 `.md`로 정리해줘.

에이전트가 “이건 프롬프트인지 스킬인지”를 분류하고, 회사 골격(역할·목적·대상·배경·제약·출력)에 맞춰 본문을 정돈합니다.

2

### 정제 — 민감정보 제거 · 출처 · frontmatter

고객정보·토큰·미공개 자료는 제거하거나 `<placeholder>` 로 치환합니다. 외부 자료 기반이면 출처/라이선스를 적고, 한 줄 설명과 frontmatter(권장)를 붙입니다.

### 3

#### 업로드 — Google Drive

프롬프트는 `AI-Knowhow/prompts/`, 스킬은 `AI-Knowhow/skills/` 에 올립니다. 커넥터가 있으면 에이전트가 직접(원형 유지를 위해 자동변환 끄기), 없으면 본문을 받아 사람이 웹에서 업로드합니다.

직접 업로드 폴더 바로가기 — 아래 링크를 열고 `.md` 를 드래그해서 올리면 됩니다.

 [AI-Knowhow / prompts ↗ 열기](#)

 [AI-Knowhow / skills ↗ 열기](#)

 [AI-Knowhow \(상위\) ↗ 열기](#)

폴더 이름으로 검색하면 동명 결과가 많이 나옵니다. 위 바로가기 링크로 여세요(에이전트는 folder ID로 범위를 좁힙니다). 공유 드라이브에 멤버로 추가돼 있어야 보입니다 — 안 보이면 관리자에게 요청하세요.

### 4

#### 이력 남기기 — GitHub 후보 브랜치

변경 이력·중복 관리를 위해 후보 브랜치에도 남깁니다. (`main` 직접 push 아님)

```
prompts → uploads/prompts/<name>.md
```

```
skills → uploads/skills/<skill-name>/SKILL.md
```

##### 가져오기 요약

- Drive / GitHub 후보에서 다운로드 또는 MD 복사
- 먼저 `.agents/externals/` 검색
- 보안검사 → 요약·충돌 확인 → 배치
- 승격 전엔 “검증 전 공유본”

##### 내보내기 요약

- 대화 → 재사용 `.md`로 추출
- 정제: 민감정보·출처·frontmatter
- Drive `AI-Knowhow/` 업로드
- GitHub `prompts / skills` 후보에 이력

## 4

## 어떤 단계에서도 — 꼭 지킬 두 가지

대화든 가져오기든 내보내기든, 예외 없이 이 둘만 지키면 안전합니다.

### 🚫 민감정보는 넣지·올리지 않기

고객 개인정보, 비밀번호·토큰, 미공개 계약·재무·법무 자료는 입력도 업로드도 금지. 꼭 써야 하면 `<고객명>`  
`>` · `<API_KEY>` 처럼 가립니다.

### 📄 외부 자료는 출처 남기기

인터넷 글·문서를 가져와 쓰면 어디서 왔는지 출처(가능하면 라이선스)를 함께 적습니다. 공유 `.md` 의 frontmatter  
`source` / `license` 에 기록하면 됩니다.

### 🛡️ 막히거나 개선 아이디어가 있으면

에이전트에게

💬 이 내용 피드백 `.md`로 정리해서 `AI-Knowhow/feedbacks`에 올리게 해줘 라고

하면, 정리·민감정보 제거를 도와 공유 폴더에 올려 줍니다. 운영자가 보고 개선합니다.

📁 `AI-Knowhow / feedbacks` ↗ 열기

회사 AI 사용 매뉴얼 — 라이프사이클 · 대화 요령 · 지식 공유. 누구나 · 어떤 OS · 어떤 AI 도구든 같은 방식으로.

출처: 번들의 `SETUP_WIZARD.md`, `docs/SHARED_FOLDER_GUIDE.md`,  
`.agents/rules/developer/spec-driven-development.md`,  
`.agents/rules/common/knowledge-sharing.md`, 프롬프트 라이브러리  
(`assets/prompts/`)와 `prompt-asset` · `external-doc-asset` 스킴을 쉬운 말로 옮긴 것입니다.

설치는 `INSTALL_GUIDE.html`, 상세 운영 규칙은 `docs/ASSET_PACKS.md`를 참고하세요.